UamiShop

Guía Completa de Ejecución de Pruebas

Monolito Modular y Microservicios

Temas Selectos de Ingeniería de Software
Maestría en Ciencias de la Computación (MCC)
Práctica 8

1 de abril de 2026

Índice

1. Descripción General del Proyecto **3**

2. Prerrequisitos **3**
2.1. Software requerido . . . 3
2.2. Verificar instalación . . . 3

3. Estructura del Proyecto **3**

4. Levantar el Stack Completo con Docker **4**
4.1. Primer arranque (build completo) . . . 4
4.2. Solución al problema de timeout en el primer build . . . 4
4.3. Verificar que todos los contenedores están corriendo . . . 4
4.4. Comandos de ciclo de vida del stack . . . 5

5. Pruebas Unitarias – Monolito Modular **5**
5.1. Ubicación y descripción de los tests . . . 5
5.2. Ejecutar todos los tests del monolito . . . 5
5.3. Ejecutar una clase de test específica . . . 5
5.4. Salida esperada . . . 6
5.5. Ver reportes de test . . . 6

6. Pruebas Unitarias – Microservicios **6**
6.1. Microservicio Catálogo (puerto 8081) . . . 6
6.2. Microservicio Órdenes (puerto 8082) . . . 6
6.3. Microservicio Ventas (puerto 8083) . . . 6

6.4. Compilar todos los microservicios de una sola vez 7
6.5. Paso 2 – Crear Categoría (Microservicio Catálogo) 7
6.6. Paso 3 – Crear Producto (Microservicio Catálogo) 8
6.7. Paso 4 – Crear Carrito (Microservicio Ventas) 8
6.8. Paso 5 – Agregar Producto al Carrito (Microservicio Ventas) 9
6.9. Paso 6 – Crear Orden (Microservicio Órdenes) 9
6.10. Paso 7 – Confirmar Orden 10
6.11. Paso 8 – Procesar Pago (activa evento RabbitMQ) 10
6.12. Paso 9 – Verificar Reducción de Stock (prueba de evento RabbitMQ) . . . 11

7. Pruebas de API del Monolito **11**
7.1. Verificar estado del monolito 11
7.2. Swagger UI del Monolito 11
7.3. Flujo básico en el monolito 11

8. Swagger UI – Documentación Interactiva **12**

9. Resumen de Resultados de Pruebas **12**
9.1. Flujo de eventos verificado 12

10. Solución de Problemas Comunes **13**

11. Comandos de Referencia Rápida **13**

1. Descripción General del Proyecto

UamiShop es un sistema de e-commerce implementado en dos arquitecturas complementarias:

- **Monolito Modular** (`practicaIS/demo`) – Spring Boot 3.1.5, puerto 8080
- **Microservicio Catálogo** (`uamishop-catalogo`) – Spring Boot 3.1.5, puerto 8081
- **Microservicio Órdenes** (`uamishop-ordenes`) – Spring Boot 3.1.5, puerto 8082
- **Microservicio Ventas** (`uamishop-ventas`) – Spring Boot 3.1.5, puerto 8083

Infraestructura compartida: MySQL 8.0 (puerto 3306) y RabbitMQ 3 (puertos 5672 / 15672). Todos los servicios se orquestan con Docker Compose desde la raíz del proyecto.

2. Prerrequisitos

2.1. Software requerido

Herramienta	Uso	Versión mínima
Java (JDK)	Compilación y ejecución local	17
Apache Maven	Gestión de dependencias y pruebas	3.8+
Docker	Contenedores de servicios	24.0+
Docker Compose	Orquestación del stack	2.20+
curl	Pruebas de API desde terminal	Cualquiera

2.2. Verificar instalación

```
java -version          # debe mostrar openjdk 17.x.x
mvn -version           # debe mostrar Apache Maven 3.x
docker --version       # debe mostrar Docker 24.x o superior
docker compose version
```

3. Estructura del Proyecto

Listing 1: Árbol principal del proyecto

```
PracticaIS-main/
          docker-compose.yml          # Orquestacion completa del
    stack
          mysql-init/
                01-init.sql             # Inicializa bases de
   datos
```

```
        practicaIS/demo/            # Monolito modular (puerto
   8080)
                pom.xml
                src/
                    main/java/...       # Codigo fuente (catalogo
   +ventas+ordenes)
                    test/java/...       # Tests unitarios (47
   tests)
        uamishop-catalogo/          # Microservicio catalogo (
   puerto 8081)
        uamishop-ordenes/           # Microservicio ordenes (
   puerto 8082)
        uamishop-ventas/            # Microservicio ventas (
   puerto 8083)
```

4. Levantar el Stack Completo con Docker

4.1. Primer arranque (build completo)

Ejecutar desde la raíz del proyecto `PracticaIS-main/`:

```
sudo docker compose down -v
sudo docker compose up --build -d
```

Nota: El primer build descarga imágenes Maven y compila los 4 servicios. Puede tardar entre 20 y 30 minutos dependiendo de la conexión a internet.

4.2. Solución al problema de timeout en el primer build

Si después del build aparece `dependency failed to start: container uamishop-rabbitmq is unhealthy`, ejecutar:

```
sudo docker compose up -d uamishop-catalogo uamishop-ventas
```

4.3. Verificar que todos los contenedores están corriendo

```
sudo docker compose ps
```

La salida esperada es:

```
NAME                STATUS
uamishop            Up (puerto 8080)
uamishop-catalogo   Up (puerto 8081)
uamishop-ordenes    Up (puerto 8082)
uamishop-ventas     Up (puerto 8083)
uamishop-mysql      Up (healthy)
uamishop-rabbitmq   Up (healthy)
```

4.4. Comandos de ciclo de vida del stack

```
# Levantar sin reconstruir (arranques subsecuentes)
sudo docker compose up -d

# Detener sin borrar datos
sudo docker compose down

# Detener Y borrar volumen de base de datos
sudo docker compose down -v

# Ver logs de un servicio especifico
sudo docker compose logs uamishop-catalogo --tail=50
sudo docker compose logs uamishop-ordenes --tail=50
sudo docker compose logs uamishop-ventas --tail=50
sudo docker compose logs uamishop --tail=50
```

5. Pruebas Unitarias – Monolito Modular

5.1. Ubicación y descripción de los tests

El monolito cuenta con **47 tests unitarios** organizados de la siguiente manera:

Clase de test	Qué prueba	Tests
`MoneyTest`	Operaciones aritméticas y validaciones de Money	14
`DomainExceptionTest`	Jerarquía de excepciones de dominio	4
`CarritoTest`	Reglas de negocio del agregado Carrito	9
`ItemCarritoTest`	Reglas del ítem del carrito	5
`CarritoServiceTest`	Servicio de carrito con mocks (Mockito)	15
Total		**47**

5.2. Ejecutar todos los tests del monolito

```
cd PracticaIS-main/practicaIS/demo
mvn test
```

5.3. Ejecutar una clase de test específica

```
# Solo MoneyTest
mvn test -Dtest=MoneyTest

# Solo CarritoServiceTest
mvn test -Dtest=CarritoServiceTest

# Solo tests de dominio (patron glob)
mvn test -Dtest="com.example.demo.ventas.domain.*"
```

5.4. Salida esperada

```
[INFO] Tests run: 14, Failures: 0, Errors: 0 - MoneyTest
[INFO] Tests run: 4,  Failures: 0, Errors: 0 -
   DomainExceptionTest
[INFO] Tests run: 15, Failures: 0, Errors: 0 - CarritoServiceTest
[INFO] Tests run: 9,  Failures: 0, Errors: 0 - CarritoTest
[INFO] Tests run: 5,  Failures: 0, Errors: 0 - ItemCarritoTest
[INFO] Tests run: 47, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

5.5. Ver reportes de test

```
# Reportes en formato XML (para CI/CD)
ls practicaIS/demo/target/surefire-reports/

# Ver reporte de un test especifico
cat practicaIS/demo/target/surefire-reports/\
com.example.demo.ventas.service.CarritoServiceTest.txt
```

6. Pruebas Unitarias – Microservicios

Los microservicios `uamishop-catalogo`, `uamishop-ordenes` y `uamishop-ventas` actualmente no tienen tests unitarios en sus directorios `src/test`. Los siguientes comandos verifican que la compilación es exitosa (BUILD SUCCESS sin errores).

6.1. Microservicio Catálogo (puerto 8081)

```
cd PracticaIS-main/uamishop-catalogo
mvn test
```

Salida esperada:

```
[INFO] No sources to compile  (en test)
[INFO] BUILD SUCCESS
```

6.2. Microservicio Órdenes (puerto 8082)

```
cd PracticaIS-main/uamishop-ordenes
mvn test
```

6.3. Microservicio Ventas (puerto 8083)

```
cd PracticaIS-main/uamishop-ventas
mvn test
```

6.4. Compilar todos los microservicios de una sola vez

```
# Desde el directorio PracticaIS-main/
for svc in practicaIS/demo uamishop-catalogo uamishop-ordenes
   uamishop-ventas; do
  echo "=== Compilando: $svc ==="
  cd $svc && mvn compile -q && echo "OK" || echo "FALLO"
  cd - > /dev/null
done
\end{lstlisting>

%==============================================================
\section{Pruebas de Integraci n -- Flujo Completo End-to-End}
%==============================================================

Estas pruebas verifican que los 4 servicios funcionan
   correctamente juntos, incluyendo la comunicaci n por RabbitMQ
   .

\subsection{Paso 1 -- Verificar que todos los servicios responden
   }

\begin{lstlisting}[style=bash]
curl -s -o /dev/null -w "%{http_code}" http://localhost:8080/v3/
   api-docs
# Esperado: 200 (Monolito)

curl -s -o /dev/null -w "%{http_code}" http://localhost:8081/api-
   docs
# Esperado: 200 (Catalogo)

curl -s -o /dev/null -w "%{http_code}" http://localhost:8082/api-
   docs
# Esperado: 200 (Ordenes)

curl -s -o /dev/null -w "%{http_code}" http://localhost:8083/api-
   docs
# Esperado: 200 (Ventas)
```

6.5. Paso 2 – Crear Categoría (Microservicio Catálogo)

```
curl -s -X POST http://localhost:8081/api/v1/categorias \
  -H "Content-Type: application/json" \
```

```
  -d '{"nombre":"Electronica","descripcion":"Productos
   electronicos"}'
```

Respuesta esperada (HTTP 201):

```
{
  "id": "deacbae3-c2e7-4b38-8583-57bc608fffd2",
  "nombre": "Electronica",
  "descripcion": "Productos electronicos",
  "categoriaPadreId": null
}
```

Guardar el ID de la categoría – se usará en el siguiente paso como `CATEGORIA_ID`.

6.6. Paso 3 – Crear Producto (Microservicio Catálogo)

```
CATEGORIA_ID="deacbae3-c2e7-4b38-8583-57bc608fffd2"

curl -s -X POST http://localhost:8081/api/v1/productos \
  -H "Content-Type: application/json" \
  -d "{
    \"nombre\": \"Laptop Lenovo\",
    \"descripcion\": \"Laptop de alta gama\",
    \"precio\": 15000.00,
    \"moneda\": \"MXN\",
    \"stock\": 10,
    \"sku\": \"LAP-001\",
    \"categoriaId\": \"$CATEGORIA_ID\"
  }"
```

Respuesta esperada (HTTP 201):

```
{
  "id": "f07afbe0-c084-4356-b198-204a0b9c6ed7",
  "nombre": "Laptop Lenovo",
  "precio": {"cantidad": 15000.0, "moneda": "MXN"},
  "stock": 10,
  "sku": "LAP-001",
  "disponible": true
}
```

Reglas de validación del SKU: debe seguir el patrón `[A-Z]{3}-\d{3}` (ejemplo: `LAP-001`).

6.7. Paso 4 – Crear Carrito (Microservicio Ventas)

```
CLIENTE_ID="550e8400-e29b-41d4-a716-446655440001"

curl -s -X POST \
  "http://localhost:8083/api/v1/carritos?clienteId=$CLIENTE_ID"
```

Respuesta esperada (HTTP 201):

```
{
  "id": {"valor": "c1765250-e648-48ef-b3fc-dbb30b997ad7"},
  "clienteId": "550e8400-e29b-41d4-a716-446655440001",
  "estado": "ACTIVO",
  "items": []
}
```

Importante: `clienteId` es un **query parameter** (no body JSON). El ID del carrito está anidado en `id.valor`.

6.8. Paso 5 – Agregar Producto al Carrito (Microservicio Ventas)

```
CARRITO_ID="c1765250-e648-48ef-b3fc-dbb30b997ad7"
PRODUCTO_ID="f07afbe0-c084-4356-b198-204a0b9c6ed7"

curl -s -X POST \
  "http://localhost:8083/api/v1/carritos/$CARRITO_ID/productos" \
  -H "Content-Type: application/json" \
  -d "{\"productoId\": \"$PRODUCTO_ID\", \"cantidad\": 2}"
```

Respuesta esperada – carrito con ítem y subtotal calculado:

```
{
  "estado": "ACTIVO",
  "items": [{
    "productoId": "f07afbe0-...",
    "cantidad": 2,
    "precioUnitario": {"cantidad": 15000.0, "moneda": "MXN"}
  }]
}
```

6.9. Paso 6 – Crear Orden (Microservicio Órdenes)

```
curl -s -X POST "http://localhost:8082/api/v1/ordenes" \
  -H "Content-Type: application/json" \
  -d '{
    "numeroOrden": "ORD-2026-001",
    "clienteId": "550e8400-e29b-41d4-a716-446655440001",
    "carritoId": "c1765250-e648-48ef-b3fc-dbb30b997ad7",
    "items": [{
      "productoId": "f07afbe0-c084-4356-b198-204a0b9c6ed7",
      "nombreProducto": "Laptop Lenovo",
      "sku": "LAP-001",
      "cantidad": 2,
      "precioUnitario": {"cantidad": 15000.0, "moneda": "MXN"}
    }],
    "direccionEnvio": {
      "nombreDestinatario": "Juan Perez",
      "calle": "Av. Principal 123",
      "ciudad": "Ciudad de Mexico",
```

```
      "estado": "CDMX",
      "codigoPostal": "06600",
      "pais": "Mexico",
      "telefono": "5512345678"
    }
  }'
```

Reglas de validación de DireccionEnvio:

- `pais` debe ser `"Mexico"` o `"México"` (solo envíos nacionales)
- `codigoPostal` debe tener exactamente 5 dígitos (`\d{5}`)
- `telefono` debe tener exactamente 10 dígitos (`\d{10}`)
- `nombreDestinatario` es obligatorio y no puede estar vacío

Respuesta esperada (estado inicial: `PENDIENTE`):

```
{
  "id": "e8c85157-f73a-4d73-a535-6a7ba37c1a67",
  "numeroOrden": "ORD-2026-001",
  "estado": "PENDIENTE",
  "total": {"cantidad": 30000.0, "moneda": "MXN"}
}
```

6.10. Paso 7 – Confirmar Orden

```
ORDEN_ID="e8c85157-f73a-4d73-a535-6a7ba37c1a67"

curl -s -X PATCH \
  "http://localhost:8082/api/v1/ordenes/$ORDEN_ID/confirmar?
   usuario=admin"
```

Importante: El parámetro `usuario` es un **query parameter**, no un body JSON.

Respuesta esperada:

```
{
  "estado": "CONFIRMADA"
}
```

6.11. Paso 8 – Procesar Pago (activa evento RabbitMQ)

```
curl -s -X PATCH \
  "http://localhost:8082/api/v1/ordenes/$ORDEN_ID/pago?referencia
   =PAY-12345"
```

Importante: El parámetro se llama `referencia` (no `referenciaPago`). Es un query parameter.

Respuesta esperada:

```
{
  "estado": "PAGO_PROCESADO",
  "resumenPago": {"estado": "APROBADO"}
}
```

6.12. Paso 9 – Verificar Reducción de Stock (prueba de evento RabbitMQ)

Este paso verifica que el evento `producto.comprado` fue recibido correctamente por el Microservicio Catálogo a través de RabbitMQ y que el stock se redujo automáticamente:

```
# Esperar 3 segundos para que RabbitMQ entregue el mensaje
sleep 3

# Verificar que el stock bajo de 10 a 8 (2 unidades compradas)
curl -s http://localhost:8081/api/v1/productos/f07afbe0-c084
   -4356-b198-204a0b9c6ed7 \
  | python3 -c "import json,sys; d=json.load(sys.stdin); print('
   Stock:', d['stock'])"
```

Resultado esperado: `Stock: 8`

Si el stock bajó de 10 a 8, el flujo completo de eventos entre microservicios funciona correctamente.

7. Pruebas de API del Monolito

El monolito expone los mismos módulos (catálogo, carrito, órdenes) en el puerto 8080.

7.1. Verificar estado del monolito

```
curl -s http://localhost:8080/v3/api-docs | python3 -c \
  "import json,sys; d=json.load(sys.stdin); print(d['info']['
   title'])"
# Salida: API de E-commerce
```

7.2. Swagger UI del Monolito

Abrir en el navegador: `http://localhost:8080/swagger-ui.html`

7.3. Flujo básico en el monolito

```
# 1. Crear categoria
curl -s -X POST http://localhost:8080/api/v1/categorias \
  -H "Content-Type: application/json" \
  -d '{"nombre":"Electronica","descripcion":"Desc"}'

# 2. Crear producto
curl -s -X POST http://localhost:8080/api/v1/productos \
  -H "Content-Type: application/json" \
  -d '{"nombre":"Mouse","sku":"MOU-001","precio":500,"moneda":"
   MXN",
       "stock":20,"categoriaId":"<ID_CATEGORIA>"}'

# 3. Crear carrito
curl -s -X POST "http://localhost:8080/api/v1/carritos?clienteId
   =<UUID>"
```

8. Swagger UI – Documentación Interactiva

Cada servicio expone documentación interactiva Swagger:

Servicio	URL Swagger	Puerto
Monolito	`http://localhost:8080/swagger-ui.html`	8080
Catálogo	`http://localhost:8081/swagger-ui.html`	8081
Órdenes	`http://localhost:8082/swagger-ui.html`	8082
Ventas	`http://localhost:8083/swagger-ui.html`	8083
RabbitMQ	`http://localhost:15672`	15672

Credenciales RabbitMQ: usuario `guest`, contraseña `guest`.

9. Resumen de Resultados de Pruebas

Componente	Tipo de prueba	Tests	Estado
Monolito	Unitarios (JUnit 5 + Mockito)	47	**PASS**
Catálogo	Compilación + BUILD	–	**PASS**
Órdenes	Compilación + BUILD	–	**PASS**
Ventas	Compilación + BUILD	–	**PASS**
Integración	Flujo end-to-end con RabbitMQ	9 pasos	**PASS**
Total		**47**	**100 %**

9.1. Flujo de eventos verificado

$$\texttt{Ventas} \xrightarrow{\texttt{producto.agregado}} \texttt{Catálogo}\ \text{(estadísticas)}$$

$$\texttt{Órdenes} \xrightarrow{\texttt{orden.creada}} \texttt{Ventas}\ \text{(completar carrito)}$$

$$\texttt{Órdenes} \xrightarrow{\texttt{producto.comprado}} \texttt{Catálogo}\ \text{(reducir stock)}$$

10. Solución de Problemas Comunes

Error	Solución
`permission denied /var/run/docker.sock`	Usar `sudo` antes de `docker compose`
`uamishop-rabbitmq is unhealthy`	Ejecutar `sudo docker compose up -d uamishop-catalogo uamishop-ventas`
`Failed to read request` en Órdenes	Verificar `pais="Mexico"`, `telefono` 10 dígitos, `codigoPostal` 5 dígitos
`Required parameter 'usuario' is not present`	El parámetro va en la URL: `?usuario=admin`
Stock no bajó después del pago	Esperar 3-5 segundos (procesamiento asíncrono de RabbitMQ)
`mysql:latest` unhealthy	El docker-compose ya usa `mysql:8.0` (corregido)

11. Comandos de Referencia Rápida

```
# === DOCKER ===
sudo docker compose up -d                    # Levantar stack
sudo docker compose down                     # Detener
sudo docker compose ps                       # Estado de
   contenedores
sudo docker compose logs <servicio> --tail=30

# === MAVEN (tests locales) ===
mvn test                                     # Todos los tests
mvn test -Dtest=CarritoServiceTest           # Un test especifico
mvn compile -q                              # Solo compilar

# === VERIFICACION DE APIS ===
curl -s http://localhost:8080/v3/api-docs    # Monolito
curl -s http://localhost:8081/api-docs       # Catalogo
curl -s http://localhost:8082/api-docs       # Ordenes
curl -s http://localhost:8083/api-docs       # Ventas

# === SWAGGER UI ===
# Monolito:  http://localhost:8080/swagger-ui.html
# Catalogo:  http://localhost:8081/swagger-ui.html
# Ordenes:   http://localhost:8082/swagger-ui.html
# Ventas:    http://localhost:8083/swagger-ui.html
# RabbitMQ:  http://localhost:15672 (guest/guest)
```